

LAB MANUAL

Bounded Workflows, Human Approval and Tool Failure

Day 3 • one lab in six parts: route, bound and scope the loop, put a person in front of the side effect, and survive a failed tool

Version 1.0 • September 2026

Environment: Cloud Shell in your sandbox GCP project • Gemini 2.5 Flash on Vertex AI (google-genai) • Cloud Shell • agent.py from the Day 2 kit

Contents

How to use this manual

Lab 2 — Bound a workflow with approval and failure handling

Part A — Workflow or agent: route the common case past the model

Part B — Bounds: a four-turn request, stopped three ways

Part C — Scoping permissions: only the tools the task needs

Part D — Approval: a card a person can judge

Part E — Tool failure: what the model is told

Part F — Retries and idempotency: lose the reply, retry, count the copies

Appendix A — agent.env: the Lab 2 switches

Appendix B — The approval card

Appendix C — The fault table

How to use this manual

This is **one lab in six parts**, A to F, run one part at a time right after the chapter of the deck that explains it. The kit is the one from Day 2: one small service, `agent.py`, that runs a Gemini model on Vertex AI in a loop. Yesterday you gave it a tool and controlled when it fired. Today you decide how much of the path the model chooses at all, put limits on the loop, put a person in front of the side effect, and make the tool fail on purpose. You edit **one file** — `agent.env`, the switches — and run the service. You write no code.

Each part flips one switch, and the printout is what you read. → `model` is what was sent (which tools, which mode — and today `task=` when the tool list depends on the task); ← `model` is what came back; ⚙️ `run` is our code running the call, or `blocked:` with the reason, or today `error:` / `failed:` when the tool fails. New lines today: `route: direct` or `route: model` from the router; `bound: ... reached` — `stopping` from the bounds; the approval card; `fault:`, `retry n/N in Ns` and `idempotency-key:` from the failure handling. The last line counts: `steps` = model calls, `calls` = tool runs, `cost` = tokens × the prices in `agent.env`.

Commands and outputs are in separate blocks

A block captioned **command** holds exactly one line to paste — the whole line, including any `SWITCH=value` typed in front of `python`. A switch typed in front of the command applies to that one run; a switch edited in `agent.env` stays until you change it back.

The block after it, captioned **example** or **expected output**, is what the run prints. Never paste an output block: it goes to the shell and fails with `syntax error near unexpected token`.

Type a message on one line, inside double quotes, exactly as printed. In Part D the run stops and waits for you to type `y` or `n` and press Enter.

Blocks captioned **example** show one complete printout from a run of the kit, so you can see every line before you read your own. The model's own words are shown as `<...>` because they differ from run to run; reservation ids are `<id>`; idempotency keys are `<key>`; token counts, costs and timings are `<n>`. The lines that matter — → `model`, `route:`, `bound:`, ⚙️ `run`, `blocked:`, `error:`, `fault:`, `retry`, `steps=` and `calls=` — are printed as the kit prints them. Blocks captioned **kit file** print a file from the kit; blocks captioned **edit** show what to put in a file.

The story for today

Meridian Financial's Bookshelf assistant can now reserve copies of a document through `bookshelf-api`, and yesterday ended with the reservation blocked, validated and tied to the session's caller — all in code. Today asks the questions a reviewer asks next. How much of the path does the model choose at all? How many steps, how long, how much money may one request cost? Which tools does a lookup request need? Who approves a reservation, and what do they see? And when `bookshelf-api` times out, answers 500 or returns rubbish — what does the model hear, and does a retry reserve twice?

So the day is about containment. Part A routes a stock question past the model. Part B bounds the loop by steps, seconds and cost. Part C sends only the tools the task needs. Part D puts a person in front of `reserve_book` with a card they can judge. Part E breaks the tool three ways and decides what the model is told. Part F loses the API's reply, retries, and counts the copies — without an idempotency key, then with one.

The same three staff profiles as yesterday, from `callers.json`: **priya** (cleared for `internal`), **daniel** (`confidential`) and **mlro** (`restricted`). Every command today runs as `daniel`.

Cost. Every run is one or more calls to the model, and the last line prints the cost from the token counts and the price lines in `agent.env`. Part B runs a four-turn request three times under a bound that stops it early; the rest is short calls. Part F rebuilds and redeploys `bookshelf-api` with Cloud Build, as on Day 0.

Set-up (once, before Part A)

Open Cloud Shell in your sandbox project, go to the Day 2 kit folder and activate the `venv`.

COMMAND — PASTE THIS LINE

```
cd ~/regai-d2-agent-kit && source ~/regai-venv/bin/activate
```

Put `tools.json` in the state Lab 2 starts from — the three tools as Day 2's Part C defined them, with `reserve_book`'s real description. The fixture is the same file as `fixtures/tools.lab1e-fixed.json`; copying it undoes any Part E edit that was left behind.

COMMAND — PASTE THIS LINE

```
cp fixtures/tools.lab2.json tools.json
```

COMMAND — PASTE THIS LINE

```
python3 -c "import json; print([t['name'] for t in json.load(open('tools.json'))])"
```

EXPECTED OUTPUT

```
['lookup_stock', 'search_bookshelf', 'reserve_book']
```

Then set the Lab 2 starting state in `agent.env`: `TOOLS=on` and `APPROVE=auto` (Day 2 may have left either at another value), and every other switch as shipped. Open the file in the Cloud Shell Editor (or `nano agent.env`); the first block — your ids and the API URL — stays as it was yesterday. Print the whole switch block — the Lab 1 and Lab 2 lines, comments stripped — and compare:

COMMAND — PASTE THIS LINE

```
sed -n '/---- Lab 1/,/---- offline/p' agent.env | grep -v '^#' | grep .
```

EXPECTED OUTPUT — THE LAB 2 STARTING STATE. FIX ANY LINE THAT DIFFERS

```
TOOLS=on
GROUNDING=off
MODE=AUTO
ALLOWED=
PARALLEL=on
VALIDATE=off
APPROVE=auto
APPROVE_CARD=full
ROUTER=off
MAX_STEPS=10
MAX_SECONDS=60
MAX_COST_USD=0.05
PRICE_INPUT_PER_M=0.30
PRICE_OUTPUT_PER_M=2.50
TOOLS_FOR_TASK=off
FAULT=none
RETURN_ERRORS=on
RETRY=0
RETRY_BACKOFF=0.5
IDEMPOTENCY=off
```

Put the library back to three copies of every document, then one run to see that everything still answers.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left?
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the answer — 3 of 3 copies>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

Three tools on the → model line, a lookup_stock round, 3 of 3. If the header shows the wrong model or an empty API URL, the first block of agent.env needs the Day 2 values back.

The whole day, in order — every step is explained in its part:

THE ORDER

```
Part A agent.env: ROUTER=on; a stock question (route: direct) and a reservation (route: model)
Part B MAX_STEPS=3 · MAX_SECONDS=5 · MAX_COST_USD=0.0003 in front of a four-turn request; --reset
Part C agent.env: TOOLS_FOR_TASK=on; a lookup that also asks to reserve; then off again; --reset
Part D agent.env: APPROVE=ask; the card, y; the card, n; APPROVE_CARD=minimal; then APPROVE=auto
Part E FAULT=timeout · FAULT=500 · FAULT=garbage · FAULT=500 RETURN_ERRORS=off
Part F redeploy bookshelf-api; FAULT=timeout-after-success RETRY=2 without and with IDEMPOTENCY=on
```

LAB 2 • DIFFICULTY ●●●

Lab 2 — Bound a workflow with approval and failure handling

Objective

Take the tool-using loop from Day 2 and fix more of its path in code, one switch at a time, until what the model still chooses is only what needs to be. By the end of Part F you have: a stock question answered with no model call; the loop stopped by a step, a time and a cost bound; a lookup request sent no `reserve_book`; a reservation held at an approval card a person can judge, approved once and declined once; three tool failures returned to the model and one kept from it; and a lost reply retried twice — once reserving two copies for one request, once reserving one, because the second time the request carried an idempotency key the API remembered.

Environment

Cloud Shell in your own sandbox project, with the Day 1 venv (`~/regai-venv`) active and the Day 2 kit folder `regai-d2-agent-kit` as you left it — `agent.env` filled in, `google-genai` and `requests` installed. Everything today is `python agent.py ...` from the kit folder; the file you edit is `agent.env`, and `tools.json` is copied from a fixture once at the start. The Day 0 `bookshelf-api` on Cloud Run must be up, and Part F redeploys it from the updated day-0 kit.

Prerequisites

- Day 2 finished: the kit runs, `reserve_book` is in `tools.json`, and you have read a printout's `→ model / ← model / ⚙️ run` lines
- Day 0's `bookshelf-api` is deployed on Cloud Run and your account can call it; you still have the day-0 kit folder or can upload the updated zip (Part F)
- The set-up above done: `fixtures/tools.lab2.json` copied over `tools.json`, the switch block at the Lab 2 starting state, the 3 of 3 run printed
- The deck's chapter for each part taught before you start that part — each part assumes its chapter

The six parts

A — Workflow or agent. `ROUTER=on`: a stock question in the shape the router knows goes straight to `bookshelf-api` — `route: direct`, no model call. A reservation goes to the model — `route: model`.

B — Bounds. `MAX_STEPS=3`, `MAX_SECONDS=5`, `MAX_COST_USD=0.0003` in front of a request that needs four turns. The loop stops before the fourth and names the bound; the reservation it made stands.

C — Scoping permissions. `TOOLS_FOR_TASK=on`: a lookup request that also asks to reserve is sent no `reserve_book`. The `→ model` line shows `task=` and which tools went.

D — Approval. `APPROVE=ask`: the card — book, copies now and after, caller — and y or n. Then `APPROVE_CARD=minimal`: a card with nothing to judge.

E — Tool failure. `FAULT=timeout, 500, garbage`: each returned to the model, which reports it. `RETURN_ERRORS=off`: the run stops and a person looks.

F — Retries and idempotency. Redeploy `bookshelf-api`. `FAULT=timeout-after-success` `RETRY=2`: two reservations for one request. `IDEMPOTENCY=on`: one, `"replayed": true`.

Part A — Workflow or agent: route the common case past the model

Goal

One switch: ROUTER=on. A stock question in the shape the router knows — how many copies of <id> are left — is answered by agent.py calling lookup_stock directly: route: direct, steps=0, no tokens, and nothing the model could have got wrong. A reservation does not match and goes to the model — route: model and the loop exactly as yesterday. Put the two last lines side by side.

Steps

1. In agent.env set ROUTER=on.

EDIT — AGENT.ENV

```
ROUTER=on
```

2. A stock question in the router's shape. Lower case, no question mark — the router accepts both, but type it as printed.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "how many copies of PRC-KYC-003 are left"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: how many copies of PRC-KYC-003 are left
route: direct (matched: how many copies of PRC-KYC-003 are left)
❗ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
answer: PRC-KYC-003 — Customer Identification and Verification Procedure: 3 of 3 copies available.
steps=0 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

route: direct (matched: ...) — the regular expression in agent.py matched, and the loop was never entered. One ❗ run lookup_stock line, an answer: line built by the code from the API's reply, and steps=0 calls=1 tokens=0 cost=\$0.000000. There is no → model line: nothing was sent to the model.

3. A reservation. The router does not match it.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
route: model
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
❗ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1,
  "copies_available": 2, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the confirmation>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

route: model, then the loop as on Day 2: → model with three tools, function_call reserve_book, ❗ run, function_response, turn 2, the confirmation. steps=2 calls=1 and a token count.

4. Write the two last lines next to each other. Same kit, same switch, two requests: which one could the model have got wrong, and what did the other one cost?

✓ Check yourself

Run 1: `route: direct`, no `→ model` line, `steps=0 calls=1 tokens=0`.

Run 2: `route: model`, then the four lines of a tool round, `steps=2 calls=1`, a token count.

You can say what the router is (a regular expression in `agent.py`, run before the loop) and what happens to a stock question it does not match (`route: model`, and the model calls `lookup_stock` itself).

Troubleshooting

Symptom	Cause	Fix
No <code>route:</code> line at all	<code>ROUTER=on</code> not saved in <code>agent.env</code>	Step 1; or type <code>ROUTER=on</code> in front of the command
Run 1 says <code>route: model</code>	The message is not in the router's shape	Type it as printed: <code>how many copies of PRC-KYC-003 are left</code>
<code>tools=[]</code> on the <code>→ model</code> line	<code>TOOLS=off</code> left from Day 2	Set-up: <code>TOOLS=on</code>
Run 2: <code>blocked: needs approval</code>	<code>APPROVE=off</code> left from Day 2's Part F	Set-up: <code>APPROVE=auto</code>
HTTP 409 ... only n copies available	The library is low	<code>python agent.py --reset</code>

Part B — Bounds: a four-turn request, stopped three ways

Goal

Three switches, each typed in front of the command for one run. The request names one document and four steps — check the stock, reserve one copy, check again, reserve one more — so the model needs four turns, one call each: lookup, reserve, lookup, reserve. The bound is checked before each model call.

`MAX_STEPS=3`: three turns, then bound: `MAX_STEPS=3 reached — stopping`; the fourth turn, the second reserve, never happens. `MAX_SECONDS=5` and `MAX_COST_USD=0.0003`: the same request, a different bound named. Read the  run `reserve_book` line of every run: it is a reservation the bound did not undo. Then `--reset`.

Steps

1. `MAX_STEPS=3`. The bound is checked before every model call; the third call is the last, and the fourth turn never happens.

COMMAND — PASTE THIS LINE

```
MAX_STEPS=3 python agent.py --user daniel "Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more"
```

EXAMPLE — MEASURED

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 2, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 2, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "caller": "daniel", "copies": 1})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "caller": "daniel", "copies": 1}) → {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 1, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 1, "replayed": false}
→ model turn 3 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 1, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 1, "copies_total": 3}
bound: MAX_STEPS=3 reached — stopping
steps=3 calls=3 tokens=1080 cost=$0.000432 elapsed=6.29s
```

Three turns — `lookup_stock`, `reserve_book`, `lookup_stock` — then bound: `MAX_STEPS=3 reached — stopping` and `steps=3 calls=3`. No model reply after the bound, because the loop stops before asking again, and no second `reserve_book`: the loop never asked for turn 4. The second lookup shows one copy fewer than the first. The printout above is a live run; yours differs only in the reservation id and the last line's counts.

2. `MAX_SECONDS=5`. The three-turn run above took about six seconds, so this bound trips on turn 2 or 3, depending on how fast the model answers; the bound line prints the time it reached.

COMMAND — PASTE THIS LINE

```
MAX_SECONDS=5 python agent.py --user daniel "Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
✱ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ model turn 3 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
✱ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 2, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 2, "copies_total": 3}
bound: MAX_SECONDS=5 reached (<n>s) — stopping
steps=3 calls=3 tokens=<n> cost=$<n> elapsed=<n>s
```

The example above is an offline run on a full library (3, then 2). In your run the `copies_available` numbers are one lower than in step 1 (1, then 0) because the library was not reset in between: step 1's reservation stands. That is the point of this part.

3. `MAX_COST_USD=0.0003`. The cost is the token counts from every response times the two price lines in `agent.env`; the three-turn run above cost \$0.000432, so this bound trips on turn 2 or 3, and the bound line prints the cost it reached. The library is at zero after two runs, so put it back first.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

COMMAND — PASTE THIS LINE

```
MAX_COST_USD=0.0003 python agent.py --user daniel "Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
✱ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
bound: MAX_COST_USD=0.0003 reached ($<n>) — stopping
steps=2 calls=2 tokens=<n> cost=$<n> elapsed=<n>s
```

4. Count the  `run reserve_book` lines across the three runs: one per run, three in all. Every one is a reservation that was made and that the bound did not, and could not, undo. Then put the library back.

COMMAND — PASTE THIS LINE
`python agent.py --reset`

EXPECTED OUTPUT
`{"status": "reset", "books": 13}`

✓ Check yourself

Three runs, three `bound: ... reached — stopping` lines, one per bound, each on the line before `steps=`.

Step 1: `lookup_stock`, `reserve_book`, `lookup_stock`, then the bound — `steps=3 calls=3` — and the fourth turn, the second reserve, never happened.

No `← model text:` after a bound: the run ended without a reply.

You counted the  `run reserve_book` lines and can say what stands after a bound hits: everything the run did before it — the bound stops the loop, it does not undo the reservation.

`--reset` printed `"status": "reset"`.

Troubleshooting

Symptom	Cause	Fix
The model asks which document or how many copies; <code>steps=1 calls=0</code> , no <code>bound:</code> line	The request was open-ended; the model declines an unbounded ask on its own	The request must name the document and the steps — type it as printed
The run ends with a model reply and no <code>bound:</code> line	The model did the four steps in fewer turns than the bound allows	Lower the bound (<code>MAX_STEPS=2</code> , <code>MAX_SECONDS=3</code> , <code>MAX_COST_USD=0.0002</code>) and rerun; use the message as printed
Step 2 ends with a reply and no <code>bound:</code> line	The model answered fast; all four turns fit in five seconds	<code>MAX_SECONDS=3</code>
Step 3 never hits the cost bound	Token counts lower than the bound	<code>MAX_COST_USD=0.0002</code>
HTTP 409 ... only 0 copies available in a  <code>run</code> line	The library ran out — each earlier run reserved a copy and no bound undid it	<code>python agent.py --reset</code> , then rerun the step

Part C — Scoping permissions: only the tools the task needs

Goal

One switch in `agent.env`: `TOOLS_FOR_TASK=on`. One message that asks two things — how many copies are left, and reserve it. `agent.py` classifies the task from the first clause (`lookup`) and sends only the read-only tools; the `→ model` line shows `task=lookup` and two names. The model reports the stock and says it cannot reserve. Switch off, same message: three tools and a reservation. Then `--reset`.

Steps

1. In `agent.env` set `TOOLS_FOR_TASK=on`. Leave `ROUTER=on` — the message below is not in the router's shape.

EDIT — AGENT.ENV

```
TOOLS_FOR_TASK=on
```

2. A lookup that also asks to reserve.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left, and reserve it for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left, and reserve it for daniel
→ model turn 1 task=lookup tools=[lookup_stock, search_bookshelf] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 task=lookup tools=[lookup_stock, search_bookshelf] mode=AUTO grounding=off
→ model text: <the stock, and the model saying it cannot reserve — it has no tool for it>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

→ `model turn 1 task=lookup tools=[lookup_stock, search_bookshelf]:reserve_book` was not sent, so no `function_call` can name it. The model looks the stock up and, in its own words, says it cannot reserve. `steps=2 calls=1`, nothing changed. The tail of the message — "and reserve it" — was an instruction the model received and could not act on.

3. Set `TOOLS_FOR_TASK=off` again, and run the same line.

EDIT — AGENT.ENV

```
TOOLS_FOR_TASK=off
```

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left, and reserve it for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left, and reserve it for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1,
  "copies_available": 2, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the stock, and the confirmation of the reservation>
steps=2 calls=2 tokens=<n> cost=$<n> elapsed=<n>s
```

Three tools, two `function_call` lines (or one per turn — either way a reservation),  `run reserve_book, calls=2`. Same request, same model; one line of code between the two outcomes.

4. Put the library back.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

✓ **Check yourself**

Run 1: `task=lookup` on the → `model` line and a two-name tool list; no  `run reserve_book, calls=1`.

Run 2: no `task=`, three names,  `run reserve_book, calls=2`.

You can say where the task was decided (the first clause of the message, in code, before the model saw it) and what a message that leads with the reservation would get (every tool — scope is per task, and Part D guards the reservation).

Troubleshooting

Symptom	Cause	Fix
No <code>task=</code> on the → <code>model</code> line	<code>TOOLS_FOR_TASK=on</code> not saved	Step 1
Run 1: <code>blocked: reserve_book is not declared in tools.json</code>	The model asked for a tool it was not sent; the loop's registry refused it	Also a correct outcome — the list decided. Note it
Run 1 reserved	The message led with the reservation, so the task was <code>reserve</code>	Type the message as printed: the lookup first
<code>route: direct</code> and no model call	The message matched the router	Type the message as printed — it has a second clause

Part D — Approval: a card a person can judge

Goal

One switch in `agent.env`: `APPROVE=ask`. A reservation request now stops at a card — action, book with its title, copies with the stock now and after, caller with name, role and clearance — and waits for `y` or `n`. Type `y`: the reservation runs. Run it again, type `n`: **blocked: declined by approver**, and the model reports it. Then `APPROVE_CARD=minimal` in front of the command: the same request, a card with nothing on it. Set `APPROVE=auto` again before Part E.

Steps

1. In `agent.env` set `APPROVE=ask`.

EDIT — AGENT.ENV

```
APPROVE=ask
```

2. A reservation. The run prints the card and waits with the cursor on the line after it. Read the card, type `y`, press Enter.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
┌ approval needed
└ action reserve_book
  book PRC-KYC-003 — Customer Identification and Verification Procedure
  copies 1 (3 available now, 2 after)
  caller daniel — Daniel Okafor, Dispute Officer, clearance confidential
└ approve? [y/n]
approval: y
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1,
  "copies_available": 2, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the confirmation>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

Every line on the card came from your systems, not from the model. `action` is the tool name from the `function_call`; `book` is the id from the model's arguments and the **title from `lookup\_stock`** — a live read of the API; `copies` is the number from the arguments and **now / after from the same read**, arithmetic in code; `caller` is the id from the arguments with **name, role and clearance from `callers.json`**. `approval: y` is what you typed. Then ⚙️ `run`, `function_response`, the confirmation.

3. The same command. This time read the `copies` line — the stock has moved — and type `n`.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
└─ approval needed ─┘
┌─ action reserve_book
│   book PRC-KYC-003 — Customer Identification and Verification Procedure
│   copies 1 (2 available now, 1 after)
│   caller daniel — Daniel Okafor, Dispute Officer, clearance confidential
└─ approve? [y/n]
approval: n
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → blocked: declined by
  approver
→ function_response reserve_book {"error": "declined by approver"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting that the reservation was declined>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

(2 available now, 1 after): the card reads the library as it is, not as the model believes it is. `approval: n`, then ⚙️ `run reserve_book(...)` → `blocked: declined by approver` and a `function_response` carrying `{"error": "declined by approver"}`. The model gets the answer, not a way past the card. `steps=2 calls=0`.

4. The bare card. `APPROVE_CARD=minimal` in front of the command; type `n`.

COMMAND — PASTE THIS LINE

```
APPROVE_CARD=minimal python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
approve reserve_book? [y/n]
approval: n
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → blocked: declined by
  approver
→ function_response reserve_book {"error": "declined by approver"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting that the reservation was declined>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

`approve reserve_book? [y/n]` and nothing else. Which book? How many? For whom? How many left after? Nothing to judge, so the answer would be a reflex — and a reflex `y` is an approval in the log with no decision behind it.

5. Set `APPROVE=auto` again before Part E, or every Part E run stops at a card first.

EDIT — AGENT.ENV

```
APPROVE=auto
```

✓ Check yourself

Run 1: the five-line card, `approval: y`, a reservation id, `calls=1`.

Run 2: the card with the stock one lower, `approval: n, blocked: declined by approver, calls=0`.

Run 3: `approve reserve_book? [y/n]` alone.

You can name, for each line of the full card, where it came from — and say which line came from the model (none; the model proposed the action, the systems described it).

`APPROVE=auto` again.

Troubleshooting

Symptom	Cause	Fix
The run does not stop at a card	<code>APPROVE=ask</code> not saved, or typed with a typo	Step 1
<code>approval: n</code> although you typed <code>y</code>	Enter pressed before <code>y</code> , or a space after it	Rerun; type <code>y</code> then Enter
The card says (<code>? available now, ? after</code>)	The model wrote a book id that is not in the catalogue	The card did its job: decline, and read the <code>function_call</code> line
Part E stops at a card	Step 5 skipped	<code>APPROVE=auto</code> in <code>agent.env</code>

Part E — Tool failure: what the model is told

Goal

Four commands, switches in front, no edits. `FAULT=timeout`, `FAULT=500`, `FAULT=garbage`: each fault is injected once in the `reserve_book` wrapper, on the first attempt; the loop turns it into an error result, and with `RETURN_ERRORS=on` (the shipped value) the model is told and reports it. Then `FAULT=500` `RETURN_ERRORS=off`: the run stops at `failed:`, nothing goes back to the model, `steps=1`, and the stop line says a person looks. Compare the four last lines.

Steps

1. A timeout. Nothing came back from the API in ten seconds — and whether it reserved is unknown.

COMMAND — PASTE THIS LINE

```
FAULT=timeout python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  fault: timeout (injected in the reserve_book wrapper)
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → error: timeout: no
  response from bookshelf-api after 10s
→ function_response reserve_book {"error": "timeout: no response from bookshelf-api after 10s"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting the timeout in its own words>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

An indented fault: `timeout (injected in the reserve_book wrapper)` line, then ⚙️ `run reserve_book(...) → error: timeout: no response from bookshelf-api after 10s`, a `function_response` carrying that text, turn 2, and the model's report. `steps=2 calls=1` — the tool run counts, though it failed. Read the model's words: does it suggest trying again?

2. An error status.

COMMAND — PASTE THIS LINE

```
FAULT=500 python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  fault: 500 (injected in the reserve_book wrapper)
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → error: HTTP 500 from
  bookshelf-api: internal error
→ function_response reserve_book {"error": "HTTP 500 from bookshelf-api: internal error"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting the 500>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

3. A body that is not JSON — an HTML error page where a reservation should be.

COMMAND — PASTE THIS LINE

```
FAULT=garbage python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  fault: garbage (injected in the reserve_book wrapper)
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → error: unparseable
  response from bookshelf-api: '<html><h1>502 Bad Gateway</h1></html>'
→ function_response reserve_book {"error": "unparseable response from bookshelf-api: '<html><h1>502 Bad
  Gateway</h1></html>'" }
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting the unreadable response>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

4. The same 500, kept from the model.

COMMAND — PASTE THIS LINE

```
FAULT=500 RETURN_ERRORS=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  fault: 500 (injected in the reserve_book wrapper)
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → failed: HTTP 500 from
  bookshelf-api: internal error
stop: not returned to the model (RETURN_ERRORS=off) — a person looks at this
steps=1 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

⚙️ run reserve_book(...) → failed: HTTP 500 ..., then stop: not returned to the model (RETURN_ERRORS=off) — a person looks at this, then steps=1 calls=1. No function_response, no turn 2, no model text. The user gets no sentence from the model; the code says what happens next, and the printout is what the person looks at.

5. Put the four last lines next to each other: three steps=2, one steps=1. For each fault, decide: after it, was the reservation made — and does the printout let you say?

✓ Check yourself

Runs 1-3: a fault: line, an error: on the ⚙️ run line, a function_response with {"error": ...}, a model report, steps=2 calls=1.

Run 4: failed:, the stop: line, no function_response, steps=1 calls=1.

You can say which of the three faults leaves the state of the reservation unknown (the timeout — and the unreadable body), and which the model could act on (a 4xx it could fix, or a plain report to the user).

Troubleshooting

Symptom	Cause	Fix
The reservation ran; no fault: line	FAULT= typed with a typo, or a value the kit does not know	timeout, 500, garbage, exactly
The run stops at an approval card	APPROVE=ask left from Part D	APPROVE=auto in agent.env
Run 4 sent the error to the model	RETURN_ERRORS=off not typed, or on in front of it	Type both switches as printed
The timeout did not take ten seconds	The fault is injected, not a real timeout	Expected — a real one waits the full HTTP_TIMEOUT

Part F — Retries and idempotency: lose the reply, retry, count the copies

Goal

First the redeploy: the updated day-0 kit's `bookshelf-api` accepts an `Idempotency-Key` header on `POST /books/{id}/reserve` — a repeated key returns the original reservation and does not reserve again. `bash deploy.sh` again, `bash verify.sh`, and the same `curl` twice to see `"replayed": true`. Then the fault: `FAULT=timeout-after-success RETRY=2` — the API reserves, the reply is lost, the retry reserves again; `lookup_stock` says 1 of 3. `--reset`; the same with `IDEMPOTENCY=on` — one key on both attempts, `"replayed": true`, `lookup_stock` says 2 of 3.

Steps — the redeploy

1. Upload `regai-day0-bookshelf-api-kit.zip` to Cloud Shell (the upload button in the toolbar) and unzip it over the day-0 folder. `-o` overwrites the files from the earlier copy of the kit.

COMMAND — PASTE THIS LINE

```
cd ~ && unzip -o -q regai-day0-bookshelf-api-kit.zip && cd day0-bookshelf-api
```

2. Set the region the service was deployed in on Day 0 — the same one, or `deploy.sh` creates a second service with a different URL — and redeploy. `deploy.sh` rebuilds the image with Cloud Build and rolls out a new revision; the service name and URL do not change, so `agent.env` stays as it is. The build takes a few minutes.

COMMAND — THE REGION FROM DAY 0

```
export REGION=asia-south1
```

COMMAND — PASTE THIS LINE

```
bash deploy.sh
```

EXPECTED OUTPUT — THE SHAPE; THE URL IS THE ONE ALREADY IN AGENT.ENV

```
project : <your project id>
region  : asia-south1

1/4 enabling APIs (about a minute the first time)...
2/4 Artifact Registry repository...
3/4 building the image with Cloud Build...
<Cloud Build output>
4/4 deploying to Cloud Run...
<Cloud Run output>

=====
bookshelf-api is deployed
URL: https://bookshelf-api-<...>.a.run.app

Save it for the labs:
export BOOKSHELF_API_URL=https://bookshelf-api-<...>.a.run.app
=====
```

3. Verify. Every line should say OK.

COMMAND — PASTE THIS LINE

```
bash verify.sh
```

EXPECTED OUTPUT

```
url      : https://bookshelf-api-<...>.a.run.app
health   : OK {"status":"ok","books":13,"chaos":"off"}
catalogue: OK (5 documents matching 'POL')
one book : POL-CR-014 | approved | confidential

done — if every line says OK, day 1 can start.
```

- The Idempotency-Key route, by hand. The same `curl` twice, same key: the second response is the first reservation again, `"replayed": true`, and `copies_available` does not move. Set the URL first so the command fits on a line.

COMMAND — PASTE THIS LINE

```
export BOOKSHELF_API_URL=$(gcloud run services describe bookshelf-api --region $REGION --format='value(status.url)')
```

COMMAND — PASTE THIS LINE

```
curl -s -X POST "$BOOKSHELF_API_URL/books/PRC-KYC-003/reserve" -H "Authorization: Bearer $(gcloud auth print-identity-token)" -H "Content-Type: application/json" -H "Idempotency-Key: demo-1" -d '{"staff_id":"daniel","copies":1}'
```

EXPECTED OUTPUT — THE FIRST TIME

```
{"id":"<id>","book_id":"PRC-KYC-003","staff_id":"daniel","copies":1,"created":"<n>","copies_available":2,"replayed":false}
```

COMMAND — PASTE THIS LINE

```
curl -s -X POST "$BOOKSHELF_API_URL/books/PRC-KYC-003/reserve" -H "Authorization: Bearer $(gcloud auth print-identity-token)" -H "Content-Type: application/json" -H "Idempotency-Key: demo-1" -d '{"staff_id":"daniel","copies":1}'
```

EXPECTED OUTPUT — THE SECOND TIME: THE SAME ID, REPLAYED, STILL 2 AVAILABLE

```
{"id":"<id>","book_id":"PRC-KYC-003","staff_id":"daniel","copies":1,"created":"<n>","copies_available":2,"replayed":true}
```

If the second response says `"replayed": false` and `copies_available` is 1, the service that answered is the old revision: check that `deploy.sh` ran in the folder you just unzipped, and that `REGION` is the region the service lives in.

Steps — the fault, without a key

- Back to the agent kit, and the library back to three copies (this also clears the API's key table).

COMMAND — PASTE THIS LINE

```
cd ~/regai-d2-agent-kit && source ~/regai-venv/bin/activate
```

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

- The fault that matters: the API does the work, then the reply is lost. `RETRY=2` retries after `RETRY_BACKOFF × 2n` seconds — 0.5 s, then 1 s.

COMMAND — PASTE THIS LINE

```
FAULT=timeout-after-success RETRY=2 python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  fault: timeout-after-success (injected in the reserve_book wrapper)
  retry 1/2 in 0.5s: timeout: no response from bookshelf-api after 10s
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>","book_id": "PRC-KYC-003", "copies": 1, "copies_available": 1, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>","book_id": "PRC-KYC-003", "copies": 1, "copies_available": 1, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the confirmation — one reservation, in the model's words>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

fault: timeout-after-success, then retry 1/2 in 0.5s: timeout: ..., then one  run line with a reservation id and "copies_available": 1, "replayed": false. The model confirms one reservation. Nothing in this printout looks wrong.

7. Count the copies.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left?
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
 run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 1, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure", "copies_available": 1, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the answer — 1 of 3 copies>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

1 of 3: two copies gone for a request that asked for one. The first attempt reserved and its reply was lost; the retry was a second request the API could not tell from the first. The duplicate is invisible from inside the run that caused it — only a read afterwards shows it.

Steps — the same fault, with a key

8. Reset, then the same command with IDEMPOTENCY=on: one key per request, made before the first attempt, sent unchanged on every retry.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

COMMAND — PASTE THIS LINE

```
FAULT=timeout-after-success RETRY=2 IDEMPOTENCY=on python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
  idempotency-key: agent-<key> (same key on every retry)
  fault: timeout-after-success (injected in the reserve_book wrapper)
  retry 1/2 in 0.5s: timeout: no response from bookshelf-api after 10s
 run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": true}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": true}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the confirmation>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

idempotency-key: agent-<key> (same key on every retry) before the fault; the same retry; and the  run line now says "copies_available": 2, "replayed": true — the API recognised the key, returned the original reservation, and did not reserve again.

9. Count the copies again.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left?
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 2, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 2, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the answer — 2 of 3 copies>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

2 of 3. Same request, same fault, same retry; one copy gone instead of two, because the second request carried the first one's name and the server remembered it.

10. Put the two ⚙️ `run reserve_book` lines from the two runs side by side: `"copies_available": 1, "replayed": false` against `"copies_available": 2, "replayed": true`. Then `--reset`, and set `ROUTER=off` in `agent.env` if you want the kit back at its shipped state.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

✓ Check yourself

`verify.sh`: OK on every line. The second `curl`: `"replayed": true`, `copies_available` unchanged. Without the key: `retry 1/2`, `"replayed": false`, and `lookup_stock` says 1 of 3.

With the key: an `idempotency-key`: line, `retry 1/2`, `"replayed": true`, and `lookup_stock` says 2 of 3.

You can say where the key is made (once per request, in `agent.py`, before the first attempt), where it is kept (`bookshelf-api`, until `--reset`), and why the client cannot make a write idempotent alone.

Troubleshooting

Symptom	Cause	Fix
The second <code>curl</code> says <code>"replayed": false</code>	The old revision is answering: <code>deploy.sh</code> ran in the old folder, or in another region	Unzip the updated kit over the folder; <code>export REGION=</code> the Day 0 region; <code>bash deploy.sh</code> again
401 / 403 from <code>curl</code> or <code>verify.sh</code>	No identity token, or your account lacks <code>roles/run.invoker</code>	<code>gcloud auth login</code> ; the day-0 README's role command
With the key: <code>"replayed": false</code> and 1 of 3	The service ignored the header — see the first row	Redeploy, then rerun from the reset
No <code>retry</code> line; the run stops at <code>failed</code> :	<code>RETRY=2</code> not typed, or misspelt	Type the switches as printed
<code>retry 1/2</code> in 1.0s or another number	<code>RETRY_BACKOFF</code> changed in <code>agent.env</code>	Fine — the wait is <code>RETRY_BACKOFF × 2ⁿ</code>
<code>deploy.sh</code> : <code>PERMISSION_DENIED</code> enabling services	Missing <code>roles/serviceusage.serviceUsageAdmin</code>	Ask the project owner, or enable the three APIs in the console

Appendix A — agent.env: the Lab 2 switches

The Lab 2 block of `agent.env.example`, verbatim. The values shown are the shipped ones, which are also the Lab 2 starting state — with `TOOLS=on` and `APPROVE=auto` from the Lab 1 block. A switch typed in front of the command (`MAX_STEPS=3 python agent.py ...`) wins over the file for that one run.

KIT FILE — AGENT.ENV.EXAMPLE, THE LAB 2 BLOCK

```
# ---- Lab 2
# A: on = a stock question matched by a regex goes straight to bookshelf-api, no model call.
ROUTER=off
# B: the loop stops and says which bound it hit. Cost = tokens x the prices below.
MAX_STEPS=10
MAX_SECONDS=60
MAX_COST_USD=0.05
# USD per million tokens for gemini-2.5-flash at the time of writing; check the price list.
PRICE_INPUT_PER_M=0.30
PRICE_OUTPUT_PER_M=2.50
# C: on = the tool list sent to the model depends on the task (a lookup gets no reserve_book).
TOOLS_FOR_TASK=off
# E: a fault injected in the reserve_book wrapper, once, on the first attempt:
#   none | timeout | 500 | garbage | timeout-after-success
#   RETURN_ERRORS: on = the error text goes back to the model; off = the run stops.
FAULT=none
RETURN_ERRORS=on
# F: retries with backoff (RETRY_BACKOFF x 2^n seconds), and an Idempotency-Key per request.
RETRY=0
RETRY_BACKOFF=0.5
IDEMPOTENCY=off
```

What each part sets, in order: **A** `ROUTER=on` in the file · **B** `MAX_STEPS`, `MAX_SECONDS`, `MAX_COST_USD` on the command line · **C** `TOOLS_FOR_TASK=on`, then `off` · **D** `APPROVE=ask`, `APPROVE_CARD=minimal` on the command line, then `APPROVE=auto` · **E** `FAULT`, `RETURN_ERRORS` on the command line · **F** `FAULT`, `RETRY`, `IDEMPOTENCY` on the command line.

Switch	Values	Part	Effect
ROUTER	off / on	A	on: a stock question the regular expression matches goes straight to <code>bookshelf-api</code> , no model call; prints <code>route: direct</code> or <code>route: model</code>
MAX_STEPS	an integer (10)	B	The loop stops before the model call that would exceed it and prints <code>bound: MAX_STEPS=n reached – stopping</code>
MAX_SECONDS	seconds (60)	B	Likewise, on wall-clock time since the run started
MAX_COST_USD	dollars (0.05)	B	Likewise, on tokens × <code>PRICE_INPUT_PER_M</code> / <code>PRICE_OUTPUT_PER_M</code>
TOOLS_FOR_TASK	off / on	C	on: the tool list depends on the task classified from the first clause; a <code>lookup</code> task gets no <code>reserve_book</code> ; the <code>→ model</code> line shows <code>task=</code>
APPROVE	auto / off / ask	D	ask: the approval card before <code>reserve_book</code> , then wait for y / n; n returns <code>{"error": "declined by approver"}</code>
APPROVE_CARD	full / minimal	D	The card with book, copies now and after, caller — or one bare line
FAULT	none / timeout / 500 / garbage / timeout-after-success	E, F	Injected in the <code>reserve_book</code> wrapper, once, on the first attempt (Appendix C)
RETURN_ERRORS	on / off	E	on: the error text goes back to the model as the <code>function_response</code> ; off: the run stops at <code>failed:</code>
RETRY	0 ... 3	F	Retries after a <code>ToolFailure</code> , each after <code>RETRY_BACKOFF × 2ⁿ</code> seconds; prints <code>retry n/N in Ns</code>
RETRY_BACKOFF	seconds (0.5)	F	The first wait; doubles on each retry
IDEMPOTENCY	off / on	F	on: one <code>Idempotency-Key</code> per request, made before the first attempt, the same on every retry; prints <code>idempotency-key:</code>

Appendix B — The approval card

What `APPROVE=ask` prints before a `reserve_book` call, and where each line comes from. The model contributes the proposal — which tool, which id, how many; every line that describes the proposal is built in `agent.py` from your systems, at the moment the card is printed.

THE FULL CARD — `APPROVE_CARD=FULL`

```
approval needed —
action    reserve_book
book      PRC-KYC-003 — Customer Identification and Verification Procedure
copies    1 (3 available now, 2 after)
caller    daniel — Daniel Okafor, Dispute Officer, clearance confidential
approve?  [y/n]
```

Line	What a judge needs it for	Where it comes from
action	what will happen	the tool name in the <code>function_call</code>
book	the object, by name — the right document?	the id from the model's arguments; the title from <code>lookup_stock</code> , a live read of <code>bookshelf-api</code>
copies	the amount, and the state before and after	the number from the arguments; available now from the same read; after = now – copies, in code
caller	who it is for, and whether they may	the id from the arguments (compared with the session on Day 2); name, role and clearance from <code>callers.json</code>
approve? [y/n]	a way to say no that the model then hears	your keyboard; <code>n</code> becomes <code>{"error": "declined by approver"}</code> in the <code>function_response</code>

THE BARE CARD — `APPROVE_CARD=MINIMAL`

```
approve reserve_book? [y/n]
```

The bare card asks the same question with nothing behind it. An approval is a decision only if the card carried enough to decide; the log cannot tell a decision from a reflex.

Appendix C — The fault table

`FAULT=` injects one failure in the `reserve_book` wrapper in `agent.py`, once, on the first attempt, so that a retry can get through. Each becomes a `ToolFailure` — the same exception a real timeout, connection error, 5xx status or unreadable body raises — and `RETURN_ERRORS` and `RETRY` decide what happens next.

<code>`FAULT=`</code>	What is injected	What the <code>`🔧 run`</code> line says	Was the reservation made?
<code>timeout</code>	no response within <code>HTTP_TIMEOUT</code> (10 s)	<code>error: timeout: no response from bookshelf-api after 10s</code>	not by the injected fault; for a real timeout, unknown
<code>500</code>	an error status from the API	<code>error: HTTP 500 from bookshelf-api: internal error</code>	not by the injected fault; for a real 500, almost certainly not
<code>garbage</code>	a body that is not JSON	<code>error: unparseable response from bookshelf-api: '<html>...'</code>	not by the injected fault; for a real one, unknown
<code>timeout-after-success</code>	the API reserves, then the reply is lost	<code>retry n/N in Ns: timeout: ... (with RETRY), then the retry's result</code>	yes — and again on each retry without a key; once with one

Then	<code>`RETURN_ERRORS=on`</code> (shipped)	<code>`RETURN_ERRORS=off`</code>
after the last attempt fails	<code>← function_response reserve_book {"error": "..."};</code> the model reports it; <code>steps=2</code>	<code>failed: ..., then stop: not returned to the model (RETURN_ERRORS=off) — a person looks at this; steps=1</code>
with <code>RETRY=n</code>	each attempt after the first waits <code>RETRY_BACKOFF × 2ⁿ</code> s and prints <code>retry</code> ; the error reaches the model only if every attempt fails	likewise; the stop comes after the last attempt
with <code>IDEMPOTENCY=on</code>	<code>idempotency-key: agent-...</code> (same key on every retry); the API replays a repeated key: <code>"replayed": true</code>	likewise

A 4xx from the API — 409 only `n` copies available, 422 copies: Input should be between 1 and 3 — is an answer, not a failure: it is returned to the model as an error result whatever the switches say, and a retry would not change it.